

1 C++ STL I

1.1 Array

Time Complexity

Operation	C++ Function	Complexity
Access	<code>arr[i]</code>	$O(1)$
Search	Manual loop	$O(n)$
Insert (end/back)	<code>arr[i] = x</code>	$O(1)$ (if space)
Insert (front)	Manual shift	$O(n)$
Insert (middle)	Manual shift	$O(n)$
Delete (end/back)	Manual (assignment)	$O(1)$
Delete (front)	Manual shift	$O(n)$
Delete (middle)	Manual shift	$O(n)$

Usage:

When you know the exact size of your data beforehand and don't need it to change. Great for fast lookups (like a constant-time lookup table) or storing small, unchanging sets of information.

Best for fixed, static data with fast random access.

Size: Fixed — cannot grow or shrink at runtime.

C++ Usage

```
// Header: None (built-in language feature)

// Description: Fixed-size block of memory. Cannot grow or shrink.
int arr[5] = {10, 20, 30, 40, 50};

// Access Element: O(1)
int element = arr[2]; // Accesses 30

// Insert: (Done by assignment, requires manual shifting for true insertion)
arr[4] = 99;

// Size: Shows the number of elements in the array
size_t size = sizeof(arr) / sizeof(arr[0]);
```

`size_t` is an unsigned integer type used for sizes and counts. It is the type returned by the `sizeof` operator and is defined in the `<cstdint>` header.

1.2 Vector

Time Complexity

Operation	C++ Function	Complexity
Access	<code>v[i], v.at(i)</code>	$O(1)$
Search	Manual loop or <code>std::find</code>	$O(n)$
Insert (end/back)	<code>v.push_back(x)</code>	Amortized $O(1)$
Insert (front)	<code>v.insert(v.begin(), x)</code>	$O(n)$
Insert (middle)	<code>v.insert(iterator, x)</code>	$O(n)$
Delete (end/back)	<code>v.pop_back()</code>	$O(1)$
Delete (front)	<code>v.erase(v.begin())</code>	$O(n)$
Delete (middle)	<code>v.erase(iterator)</code>	$O(n)$

Usage:

The default choice for a list of items where you mainly add new elements to the end, but still need fast access to any item by its index (like standard Python lists or JavaScript arrays).

Best when you need dynamic size and frequent access at arbitrary indices.

Size: Dynamic — resizable at runtime.

C++ Usage

```
#include <vector>

// Description: A dynamic, resizable array.
std::vector<int> v = {1, 2, 3};

// Insert (End/Back): Amortized O(1)
v.push_back(4);

// Access Element: O(1)
int element_3 = v[2]; // 3

// Remove (End/Back): O(1)
v.pop_back(); // Removes 4

// Insert (Middle): O(n) - requires shifting
v.insert(v.begin() + 1, 99); // Inserts 99 at index 1

// Remove (Middle): O(n) - requires shifting
v.erase(v.begin() + 1); // Removes 99

// Size and Checks
size_t current_size = v.size();
bool is_empty = v.empty();
```

1.3 Deque

Time Complexity

Operation	C++ Function	Complexity
Access	<code>dq[i]</code> , <code>dq.at(i)</code>	$O(1)$
Search	Manual loop or <code>std::find</code>	$O(n)$
Insert (end/back)	<code>dq.push_back(x)</code>	$O(1)$
Insert (front)	<code>dq.push_front(x)</code>	$O(1)$
Insert (middle)	<code>dq.insert(iterator, x)</code>	$O(n)$
Delete (end/back)	<code>dq.pop_back()</code>	$O(1)$
Delete (front)	<code>dq.pop_front()</code>	$O(1)$
Delete (middle)	<code>dq.erase(iterator)</code>	$O(n)$

Usage:

When you need fast operations (insert/remove) at both the beginning and the end of the list, such as implementing a double-ended queue or managing a sliding window in an algorithm.

Best when you need fast push/pop from both ends.

Size: Dynamic — resizable at runtime.

C++ Usage

```
#include <deque>

// Description: Double-Ended Queue. Efficient for front and back operations.
std::deque<int> dq = {1, 2, 3};

// Insert (Front): O(1)
dq.push_front(0); // dq is now {0, 1, 2, 3}

// Insert (Back): O(1)
dq.push_back(4); // dq is now {0, 1, 2, 3, 4}

// Remove (Front): O(1)
dq.pop_front(); // Removes 0

// Remove (Back): O(1)
dq.pop_back(); // Removes 4

// Access Element (Random Access): O(1)
int element_1 = dq[1]; // Accesses 2
```

1.4 Stack

Time Complexity

Operation	C++ Function	Complexity
Access (top)	<code>st.top()</code>	$O(1)$
Search	Manual iteration (destructive)	$O(n)$

Queue

Operation	C++ Function	Complexity
Insert (push)	<code>st.push(x)</code>	$O(1)$
Delete (pop)	<code>st.pop()</code>	$O(1)$

Usage:

Modeling situations where the last thing you add is the first thing you deal with (LIFO). Think of browser history (undo/redo), simulating recursion, or checking if parentheses match in an expression.

LIFO problems like parsing, recursion, backtracking.

Size: Dynamic — size changes at runtime (depends on the underlying container used).

C++ Usage

```
#include <stack>

// Description: LIFO (Last-In, First-Out) container adaptor.
std::stack<int> st;

// Insert (Push): O(1)
st.push(5);
st.push(10); // 10 is now on top

// Access Top Element: O(1)
int top_val = st.top(); // 10

// Remove Top Element: O(1) (Does not return value)
st.pop(); // Removes 10

// Checks
bool is_empty = st.empty();
```

1.5 Queue

Time Complexity

Operation	C++ Function	Complexity
Access (front/back)	<code>q.front()</code> , <code>q.back()</code>	$O(1)$
Search	Manual iteration (destructive)	$O(n)$
Insert (enqueue)	<code>q.push(x)</code>	$O(1)$
Delete (dequeue)	<code>q.pop()</code>	$O(1)$

Usage:

Modeling situations where the first thing you add is the first thing you deal with (FIFO). Standard use cases include managing a list of tasks, scheduling jobs, or performing Breadth-First Search (BFS) in a graph.

FIFO problems like task scheduling, level-order traversal.

Size: Dynamic — size changes at runtime (depends on the underlying container used).

C++ Usage

```
#include <queue>

// Description: FIFO (First-In, First-Out) container adaptor.
std::queue<int> q;

// Insert (Push to back): O(1)
q.push(7);
q.push(8); // 7 is at the front

// Access Front Element: O(1)
int front_val = q.front(); // 7

// Access Back Element: O(1)
int back_val = q.back(); // 8

// Remove Front Element (Pop): O(1) (Does not return value)
q.pop(); // Removes 7
```

1.6 Priority Queue

Time Complexity

Operation	C++ Function	Complexity
Access (top)	pq.top()	$O(1)$
Search	Manual iteration (destructive)	$O(n)$
Insert	pq.push(x)	$O(\log n)$
Delete	pq.pop()	$O(\log n)$

Usage:

When you need to constantly pull out the ‘most important’ item first, rather than the oldest or newest item. Essential for algorithms like Dijkstra’s (finding the shortest path) or any task scheduling where priorities matter.

When elements must be processed in priority order, not FIFO.

Size: Dynamic — resizable at runtime; implemented as a heap over an underlying container.

C++ Usage

```
#include <queue>
#include <vector>
#include <functional> // Required for std::greater

// Description: Max-heap implementation by default. Highest element is
↪ always on top.
```

```
// Max-Heap (Default)
std::priority_queue<int> pq_max;
pq_max.push(30);
pq_max.push(10);

// Access Highest Priority Element: O(1)
int max_val = pq_max.top(); // 30

// Remove Highest Priority Element: O(log n) (Does not return value)
pq_max.pop();

// Min-Heap (Custom Comparator needed)
std::priority_queue<int, std::vector<int>, std::greater<int>> pq_min;
pq_min.push(30);
pq_min.push(10);
int min_val = pq_min.top(); // 10
```

References:

1.7 Summary

1.7.1 Time Complexity

Operation	Array	Vector	Deque	Stack	Queue	Priority Queue
Access (Random)	$O(1)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Access (Top/End)	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Search	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Insert (End/Push)	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(\log n)$
Insert (Front)	$O(n)$	$O(n)$	$O(1)$	-	-	-
Insert (Middle)	$O(n)$	$O(n)$	$O(n)$	-	-	-
Delete (End/Pop)	$O(1)$	$O(1)$	$O(1)$	$O(1)$	-	$O(\log n)$
Delete (Front)	$O(n)$	$O(n)$	$O(1)$	-	$O(1)$	-
Delete (Middle)	$O(n)$	$O(n)$	$O(n)$	-	-	-
Size	Fixed	Dynamic	Dynamic	Dynamic	Dynamic	Dynamic

1.7.2 C++ Usage

Data Structure	Header	Insert	Access	Delete	Ordering
Array	-	<code>arr[i] = x</code>	<code>arr[i]</code>	Manual shift	Indexed
Vector	<code><vector></code>	<code>v.push_back(x)</code>	<code>v[i]</code> , <code>v.back()</code>	<code>v.pop_back()</code> , <code>v.erase()</code>	Indexed
Deque	<code><deque></code>	<code>dq.push_front(x)</code> , <code>dq.push_back(x)</code>	<code>dq.front()</code> , <code>dq.back()</code> , <code>dq[i]</code>	<code>dq.pop_front()</code> , <code>dq.pop_back()</code>	Double-Ended
Stack	<code><stack></code>	<code>st.push(x)</code>	<code>st.top()</code>	<code>st.pop()</code>	LIFO
Queue	<code><queue></code>	<code>q.push(x)</code>	<code>q.front()</code> , <code>q.back()</code>	<code>q.pop()</code>	FIFO
Priority Queue	<code><queue></code>	<code>pq.push(x)</code>	<code>pq.top()</code>	<code>pq.pop()</code>	Priority (Heap)

1.8 References:

- [C++ Cheat Sheet](#)
- [Big O Cheat Sheet](#)